

# 中山大学计算机学院

## 人工智能

### 本科生实验报告

课程名称: Artificial Intelligence

|    |          |    |     |
|----|----------|----|-----|
| 学号 | 22336203 | 姓名 | 沈苇杭 |
|----|----------|----|-----|

#### 一、实验题目

**Lab7-3: 卷积神经网络:** 使用 pytorch 框架搭建神经网络实现中药图片分类, 利用训练集完成网络训练, 画出模型训练过程的 loss 曲线和准确率曲线。

#### 二、实验内容

##### 1. 算法原理

- 卷积神经网络:** 在计算机中, 图像按照 RGB 颜色模型来表示。这个三维矩阵中的每个数字, 表示对应颜色光的亮度。在卷积神经网络中, 卷积操作是指将一组固定权重 (卷积核) 与图像进行逐元素相乘并相加, 以此提取图像的特征。卷积神经网络由输入层 (接受原始数据), 卷积和激活层 (将输入图像与卷积核进行卷积操作, 然后通过应用激活函数来引入非线性), 池化层 (减小特征图的大小, 以此简化运算), 全连接层 (将提取的特征映射转化为输出)。
- Torch.nn:** torch.nn.Conv2d 函数: in\_channels 和 out\_channels 表示输入和输出通道数, kernel\_size 表示卷积核尺寸, stride 表示卷积步长, padding 表示图像边缘填充的方式。
- 网络训练:** 网络训练的步骤一般包括: 实例化网络, 得到 loss, 定义网络优化器。要对优化器进行更新时, 先清除上一步的梯度, 然后 loss 反向传播, 再更新优化器。
- 卷积神经网络实现图片分类:**
  - 读取图片, 进行数据预处理, 加载图片。
  - 搭建神经网络, 包括卷积层和激活层, 池化层和全连接层。
  - 将训练集中的图片按批次放入神经网络, 得到预测结果, 计算 loss 值, 进行梯度下降。
  - 每隔一段批次进行一次测试。
  - 画出 loss 和测试集准确率曲线图。

##### 2. 关键代码展示 (可选)

- 文件处理:**

创建新的 test 文件夹, 将现在的 test 图片按照标签放入新 test 文件夹中对应的文件夹, 方便 Imagefolder 处理。

```
# 现在 test 文件的形式不方便处理, 将 test 中的图片全部提取出来, 并整理成好处理的形式
```

```

old_test = "./test"
new_test = "./new_test"
# 如果目标文件 new_Test 不存在，则创建
os.makedirs("./new_test", False)
# 创建每个类的文件
os.makedirs("./new_test/baihe")
os.makedirs("./new_test/dangshen")
os.makedirs("./new_test/gouqi")
os.makedirs("./new_test/huaihua")
os.makedirs("./new_test/jinyinhua")
# 遍历旧测试集中的文件，转移到对应的类中
for filename in os.listdir(old_test):
    source_path = os.path.join(old_test, filename)
    if "baihe" in filename:
        dest_path = "./new_test/baihe"
    elif "dangshen" in filename:
        dest_path = "./new_test/dangshen"
    elif "gouqi" in filename:
        dest_path = "./new_test/gouqi"
    elif "huaihua" in filename:
        dest_path = "./new_test\huaihua"
    elif "jinyinhua" in filename:
        dest_path = "./new_test/jinyinhua"
    dest_path = os.path.join(dest_path, filename)
    shutil.move(source_path, dest_path)

```

ii. 图片处理:

将训练集和测试集分开处理，裁剪为统一尺寸，加载进 dataloader.

```

# 读取训练集
train_dir = "./train"
# 裁剪图片为统一尺寸
train_transform = transforms.Compose([transforms.Resize([64, 64]),
transforms.ToTensor()])
# 加载训练集
train_dataset = datasets.ImageFolder(root="./train", transform=train_transform)
train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
batch_size=40, shuffle=True)

# 读取测试集
test_dir = "./new_test"
test_transform = transforms.Compose([transforms.Resize([64, 64]),
transforms.ToTensor()])
# 加载测试集
test_dataset = datasets.ImageFolder(root="./new_test",

```

```
transform=test_transform)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=10,
shuffle=False)
```

iii. 搭建模型:

```
#模型
class Model(torch.nn.Module):
    def __init__(self):
        super(Model, self).__init__()
        self.Conv = torch.nn.Sequential( # 输入 3*64*64
            torch.nn.Conv2d(
                in_channels=3,
                out_channels=64,
                kernel_size=3,
                stride=1,
                padding=1
            ),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=2) # 输出 64*32*32
        )
        self.Conv1 = torch.nn.Sequential(
            torch.nn.Conv2d(
                in_channels=64,
                out_channels=128,
                kernel_size=3,
                stride=1,
                padding=1
            ),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=2) # 输出 128*16*16
        )
        self.out = torch.nn.Sequential(
            torch.nn.Linear(128 * 16 * 16, 512),
            torch.nn.ReLU(),
            torch.nn.Dropout(p=0.5),
            torch.nn.Linear(512, 5)
        )

# 前向传播
    def forward(self, x):
        x = self.Conv(x)
        x = self.Conv1(x)
        x = x.view(x.size(0), -1)
```

```
out = self.out(x)
return out, x
```

- iv. 网络训练:  
记录 loss, 准确率, 运行时间

```
# 遍历次数
traverse_time = 30
# 开始时间
time_start = time.time()
# loss
loss_function = torch.nn.CrossEntropyLoss()
# optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

Use_gpu = torch.cuda.is_available()
if Use_gpu:
    model = model.cuda()

for traverse in range(traverse_time):
    # 如果是训练模式, 就进行训练; 否则不进行训练
    for batch, data in enumerate(train_loader, 0):
        if batch % 40 != 0:
            # print("train")
            model.train(True)
            X, Y = data
            # 放到 GPU 上
            X = X.cuda()
            Y = Y.cuda()
            # 搭建模型
            output = model(X)[0]
            # 得到标签值
            predict = torch.max(output, 1)[1]
            # 收回到 CPU 上
            predict = predict.cpu().numpy()
            # 梯度归零
            optimizer.zero_grad()
            # 计算损失
            loss = loss_function(output, Y)
            loss_list.append(loss.item())
            # 反向传播
            loss.backward()
            # 梯度更新
            optimizer.step()
            # 计算 loss
```

```

        loss_total = loss_total + loss.data.item()
        correct = correct + (predict ==
Y.data.detach().cpu().numpy()).astype(int).sum().item()
        accuracy = float(correct / Y.size(0))
        acc_list.append(accuracy)
        correct = 0
    else:
        print("test")
        model.train(False)
        for batch_test, data_test in enumerate(test_loader, 0):
            X, Y = data_test
            # 放到 GPU 上
            X = X.cuda()
            Y = Y.cuda()
            # 搭建模型
            output = model(X)[0]
            # 得到标签值
            predict = torch.max(output, 1)[1]
            # 收回到 CPU
            predict = predict.cpu().numpy()
            # 梯度归零
            optimizer.zero_grad()
            # 计算损失
            loss = loss_function(output, Y)
            # 计算 loss
            loss_total = loss_total + loss.data.item()
            accuracy = float((predict ==
Y.data.detach().cpu().numpy()).astype(int).sum()) / float(Y.size(0))
            acc_list.append(accuracy)
            print('Epoch: ', traverse, ' test accuracy:', accuracy)
            print(acc_list)
time_end = time.time() - time_start
print(time_end)

```

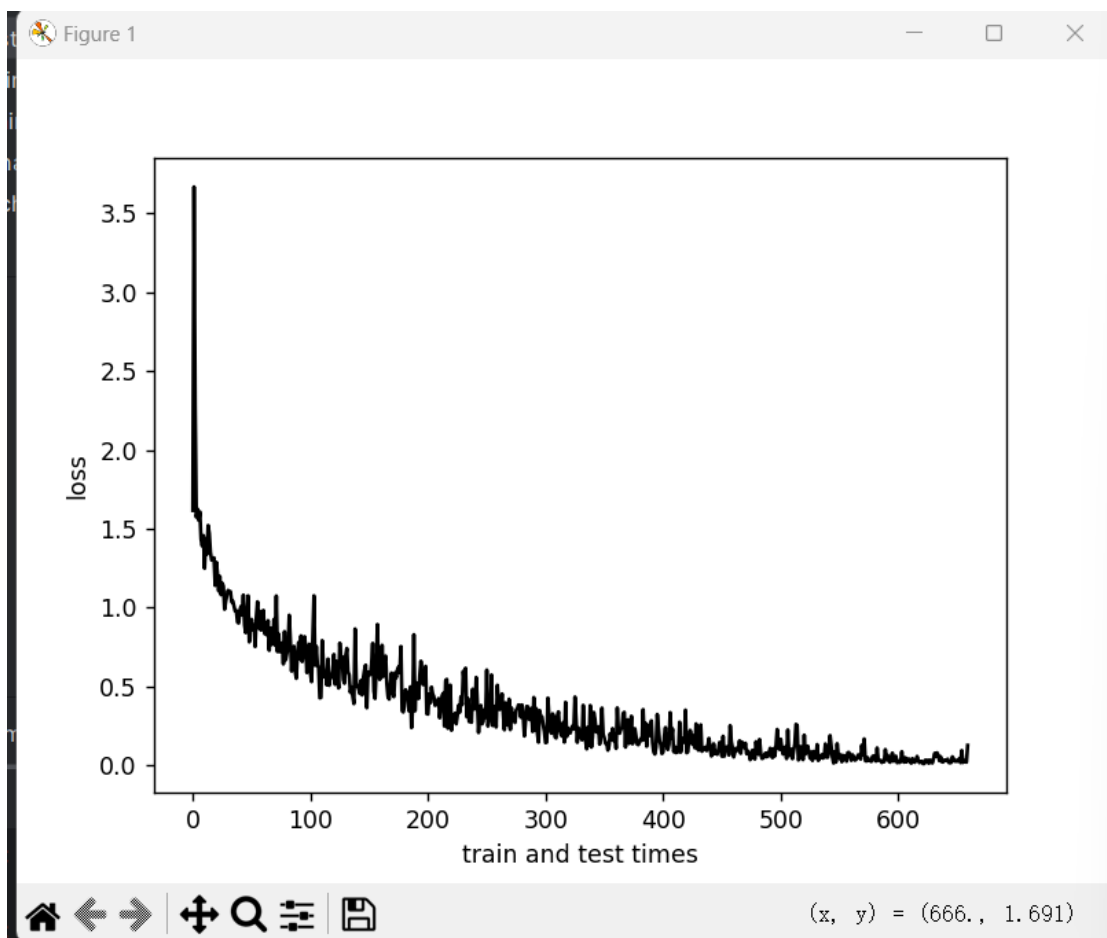
### 3. 创新点&优化（如果有）

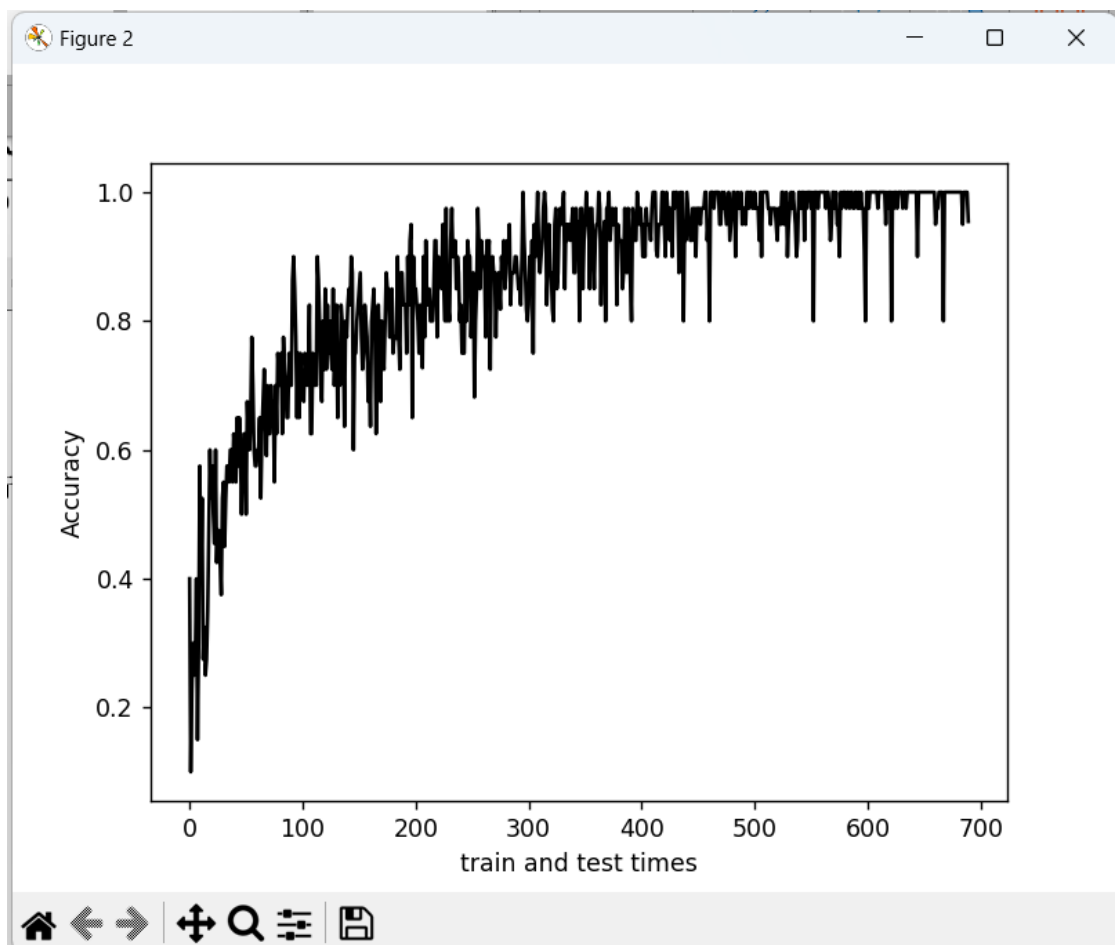
在模型的全连接层中加入了 Dropout 层，可以随机丢弃一部分神经元，起到防止过拟合的作用。

## 三、 实验结果及分析

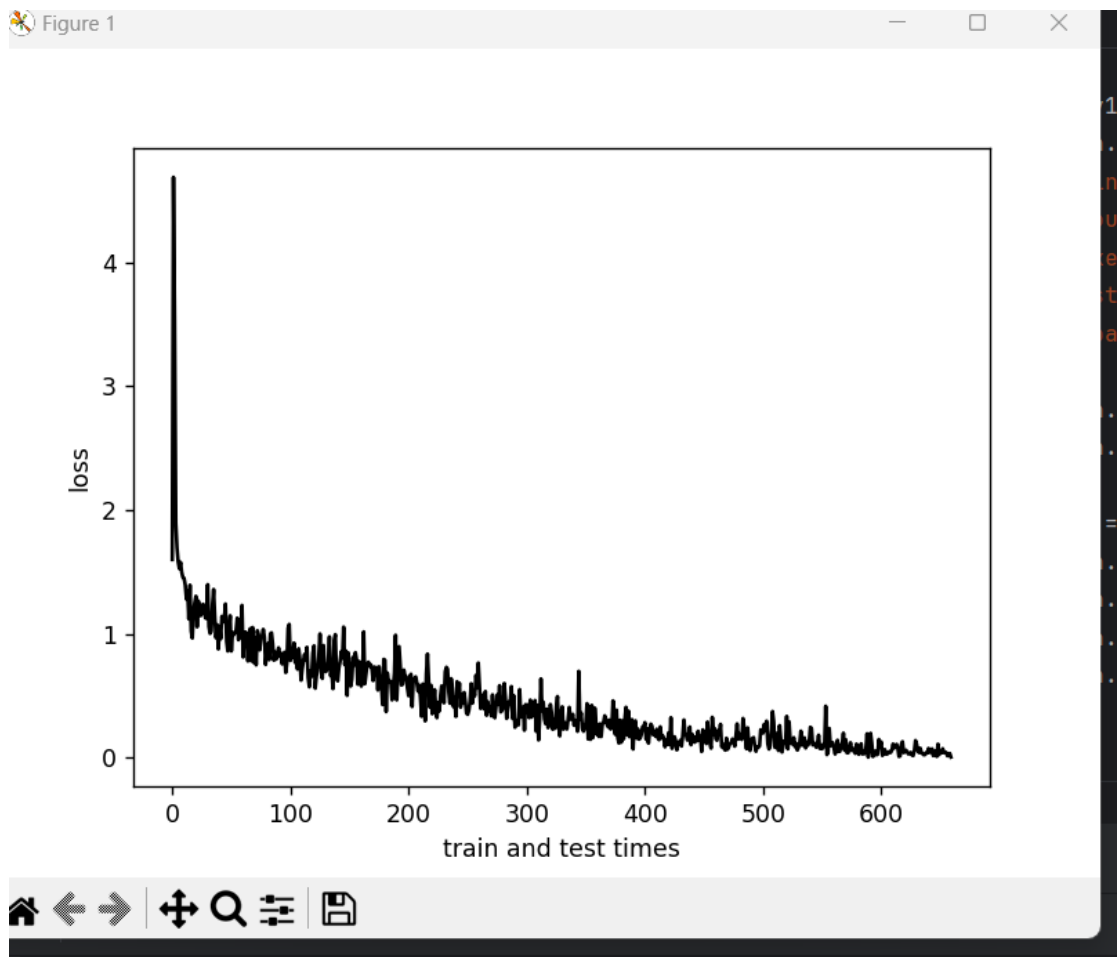
### 1. 实验结果展示示例（可图可表可文字，尽量可视化）

优化前：

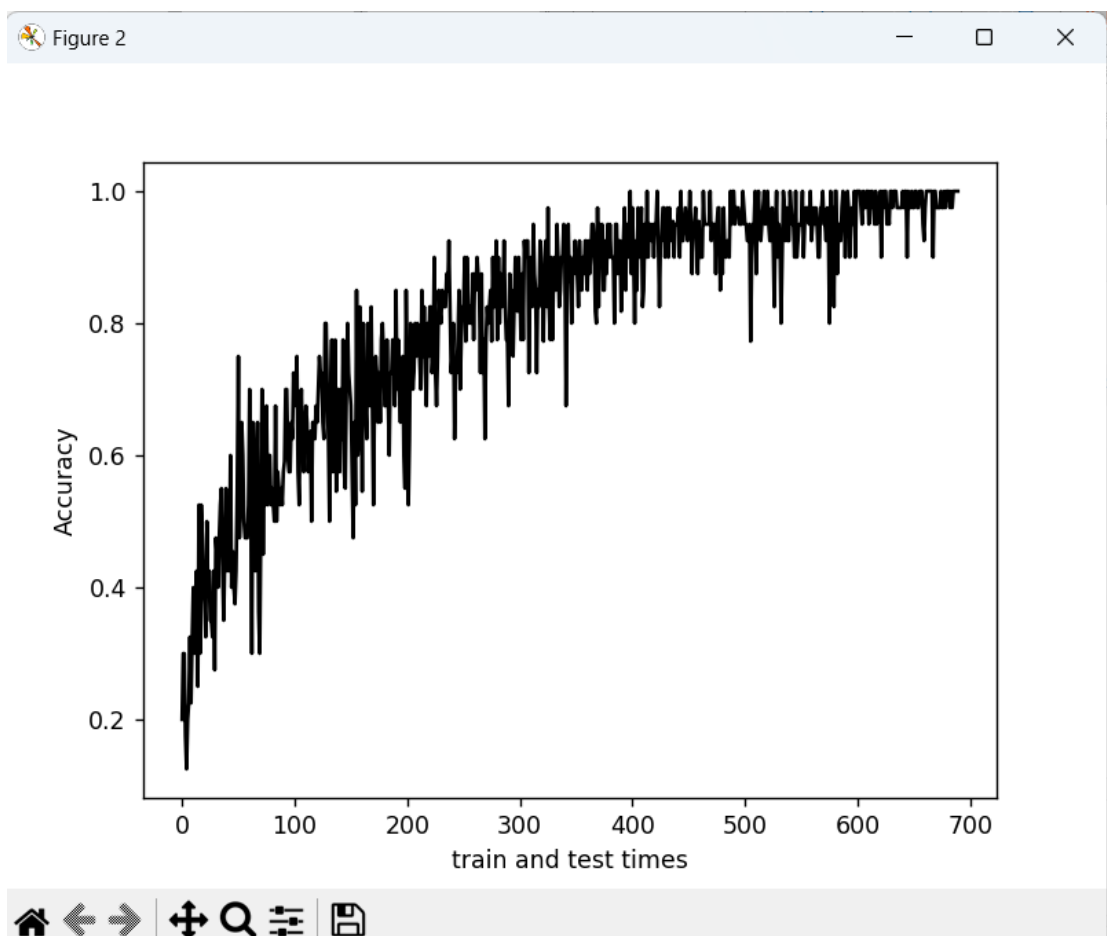




优化后：







## 2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

优化前的运行时间和最终测试准确率：

```
run time: 200.38980627059937
final_test_accuracy: 0.9 (correct= 9 )
```

优化后的运行时间和最终测试准确率：

```
run time: 194.3778042793274
final_test_accuracy: 0.9 (correct= 9 )
```

经过优化，在不影响分类准确率的前提下，缩短了 30 个 epoch 的运行时间。

## 四、 参考资料

1. [PyTorch 学习笔记 我这一次的博客-CSDN 博客](#)
2. [【深度学习】一文搞懂卷积神经网络\(CNN\)的原理\(超详细\)\\_卷积神经网络原理-CSDN 博客](#)
3. [从零开始安装 pytorch, 并在 pycharm 中使用 pytorch 和 pycharm-CSDN 博客](#)

4. [PyTorch 详细常用图像数据集加载及预处理（三种）\\_torch 如何加载本地图像数据集-CSDN 博客](#)
5. [PyTorch-Tutorial/tutorial-contents/401\\_CNN.py at master · MorvanZhou/PyTorch-Tutorial \(github.com\)](#)
6. [【pytorch】卷积操作原理解析与 nn.Conv2d 用法详解-CSDN 博客](#)
7. 理论课 PPT